

Postman Testing Guide — Setu Payment Service

Introduction to the Postman Testing Suite :

Welcome to the Postman Testing Guide for the Setu Payment Lifecycle & Reconciliation Service. Because financial systems rely heavily on complex state machines and asynchronous events, simply checking if an endpoint returns a 200 OK is not enough. This testing suite is designed to prove that the core business logic of the system functions flawlessly under real-world fintech conditions.

By following this guide, you will manually simulate the entire lifecycle of a payment. You will not only execute the **"Happy Path"** (where a payment flows smoothly from initiation to settlement), but you will also intentionally trigger network retries to prove the system's idempotency, and inject broken event sequences to witness the reconciliation engine successfully catch financial anomalies in real-time.

What this guide will prove:

1. **Safety:** Duplicate events are ignored without throwing errors or creating duplicate database rows.
2. **State Derivation:** The system correctly rolls up individual events into a master transaction status.
3. **Anomaly Detection:** The system successfully flags impossible states (like money settling after a payment has failed).

Important Endpoints and URLs :

Item	Details
Version	2.0
Status	Updated with all latest changes
Base URL	https://setu-payment-service.vercel.app
Swagger UI	https://setu-payment-service.vercel.app/docs
Developer Guide (ReDoc)	https://setu-payment-service.vercel.app/redoc
Visual Portal	https://setu-payment-service.vercel.app/

What We're Testing :

This service is a payment lifecycle event ingestion and reconciliation system. Think of it like a financial ledger that:

1. **Accepts events** (payment started, payment done, payment failed, money settled)
2. **Tracks the current state** of each transaction
3. **Detects anomalies** between what was paid vs. what was settled

The Full Event Lifecycle

```
payment_initiated → payment_processed → settled      ✓ Happy Path
payment_initiated → payment_failed                    ✗ Failure Path
```

The 7 Endpoints and its functionalities :

Serial Number	Endpoint	Method	What It Does
1	/	GET	Visual dashboard portal
2	/health	GET	JSON health check
3	/events	POST	Ingest a payment lifecycle event
4	/events	GET	List events with filters & pagination
5	/transactions	GET	List transactions with filters & pagination
6	/transactions/{id}	GET	Fetch full details + event history for one transaction
7	/reconciliation/summary	GET	Financial summary by merchant, status, date
8	/reconciliation/discrepancies	GET	Anomaly detection — transactions with inconsistencies

STEP 0 — Setup Postman

Step	Action	Details
1	Open Postman	Launch the Postman application.
2	Create Environment	Click New → Environment.
3	Name Environment	Enter Setu Payment Service.
4	Add Variable	Add the variable baseUrl.
5	Set Initial Value	https://setu-payment-service.vercel.app
6	Set Current Value	https://setu-payment-service.vercel.app
7	Save Environment	Click Save.
8	Select Environment	From the top-right dropdown, select Setu Payment Service.
9	Configure POST Headers	For every POST request, open the Headers tab.
10	Add Content-Type	Set Key: Content-Type and Value: application/json.
11	Use Base URL	Use {{baseUrl}} in your API requests, for example: {{baseUrl}}/payments.

TEST 1 : Health Check (Start Here)

Purpose: Confirm the API is alive. Always do this first.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/health
Expected Status	200 OK
Expected Response	Successful health-check response confirming that the service is running.
Pass Condition	"status" is "healthy". If this fails, the service is down — stop here.

Sample Response (200 OK) :

```
{
  "service": "Setu Payment Service",
  "version": "1.0.0",
  "status": "healthy",
  "docs": "/docs"
}
```

TEST 2 : Ingest a New Event (Happy Path — Step 1 of 3)

Purpose: Prove a brand-new payment event is accepted and stored.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events
Headers	Content-Type: application/json
Body Type	raw → JSON
Request Body	JSON payload
Pass Condition	" ingestion_status " is " created ". Note that the amount is returned as a string (not a float) — this is intentional for financial precision.

Sample Request Body for TEST 1 :

```
{
  "event_id": "test-evt-initiated-001",
  "event_type": "payment_initiated",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "merchant_name": "QuickMart",
  "amount": 7500.00,
  "currency": "INR",
  "timestamp": "2024-06-15T10:00:00Z"
}
```

Sample ResponseBody for TEST 1 (200 OK):

```
{
  "event_id": "test-evt-initiated-001",
  "event_type": "payment_initiated",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "amount": "7500.00",
  "currency": "INR",
  "timestamp": "2024-06-15T10:00:00Z",
  "received_at": "<server timestamp>",
  "ingestion_status": "created"
}
```

Field References :

Field	Type	Required	Notes
event_id	string	✓	Must be globally unique (use a UUID in production)
event_type	enum	✓	One of: payment_initiated, payment_processed, payment_failed, settled
transaction_id	string	✓	Groups multiple events into one transaction
merchant_id	string	✓	e.g. merchant_1 through merchant_5
merchant_name	string	✓	Human-readable name
amount	number or string	✓	Must be positive
currency	string	✗	Defaults to "INR"
timestamp	ISO 8601 datetime	✓	When the event occurred in the payment system

TEST 3 : Idempotency (The Most Important Test)

Purpose: Prove that sending the exact same event twice is safe — no duplicates, no errors.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events
Body	Same JSON as Test 2 — do not change anything.
Expected Status Code	200 OK
Expected ingestion_status	"already_processed"
Pass Condition	Status code remains 200 (not 400 or 500) and "ingestion_status" is "already_processed" .

Running this test successfully proves that :

- No duplicate row was created in the database
- The service is safe to retry under network failure
- This is fintech-grade reliability

Response for TEST 3 (200 OK) :

```
{
  "event_id": "test-evt-initiated-001",
  "event_type": "payment_initiated",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "amount": "7500.00",
  "currency": "INR",
  "timestamp": "2024-06-15T10:00:00Z",
  "received_at": "<original server timestamp>",
  "ingestion_status": "already_processed"
}
```

TEST 4 : Progress the Transaction (Happy Path — Step 2 of 3)

Purpose: Move test-txn-001 from initiated → processed.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events

Key Observation
Same transaction_id (test-txn-001) but a different event_id and event_type. This is how the system links multiple events into one transaction journey.

Sample Request Body for TEST 4 :

```
{
  "event_id": "test-evt-processed-001",
  "event_type": "payment_processed",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "merchant_name": "QuickMart",
  "amount": 7500.00,
  "currency": "INR",
  "timestamp": "2024-06-15T10:05:00Z"
}
```

Sample Response Body for TEST 4 (200 OK) :

```
{
  "event_id": "test-evt-processed-001",
  "event_type": "payment_processed",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "amount": "7500.00",
  "currency": "INR",
  "ingestion_status": "created"
}
```

TEST 5 : Settle the Transaction (Happy Path — Step 3 of 3)

Purpose: Complete the happy path: initiated → processed → settled.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events

Sample Request Body for TEST 5 :

```
{
  "event_id": "test-evt-settled-001",
  "event_type": "settled",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "merchant_name": "QuickMart",
  "amount": 7500.00,
  "currency": "INR",
  "timestamp": "2024-06-15T12:00:00Z"
}
```

Sample Response Body for TEST 5 (200 OK) :

```
{
  "event_id": "test-evt-settled-001",
  "event_type": "settled",
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "amount": "7500.00",
  "currency": "INR",
  "ingestion_status": "created"
}
```

TEST 6 : Test the Failure Path (Alternative Scenario)

Purpose: Prove a failed payment is tracked correctly. We will create a brand new transaction (**test-txn-002**) and intentionally fail it.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events
Headers	Content-Type: application/json
Body Type	raw → JSON
Request Body	JSON payload
Pass Condition	Both return ingestion_status: " created ". After Step 6B, the transaction's status will be "failed".

Step 6A : Initiate the Payment

```
{
  "event_id": "test-evt-fail-init-001",
  "event_type": "payment_initiated",
  "transaction_id": "test-txn-002",
  "merchant_id": "merchant_2",
  "merchant_name": "TechStore",
  "amount": 15000.00,
  "currency": "INR",
  "timestamp": "2024-06-15T11:00:00Z"
}
```

Step 6B : Fail the Payment

```
{
  "event_id": "test-evt-fail-end-001",
  "event_type": "payment_failed",
  "transaction_id": "test-txn-002",
  "merchant_id": "merchant_2",
  "merchant_name": "TechStore",
  "amount": 15000.00,
  "currency": "INR",
  "timestamp": "2024-06-15T11:02:00Z"
}
```

TEST 7 : Verify Transaction State (After Full Journey)

Purpose: Prove the state machine correctly tracked the full journey of test-txn-001.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions/test-txn-001
Pass Condition	Status is "settled" and the events array has exactly 3 entries in chronological order.

Sample Response Body for TEST 7 (200 OK) :

```
{
  "transaction_id": "test-txn-001",
  "merchant_id": "merchant_1",
  "merchant_name": "QuickMart",
  "amount": "7500.00",
  "currency": "INR",
  "status": "settled",
  "payment_status": "processed",
  "settlement_status": "settled",
  "created_at": "2024-06-15T10:00:00Z",
  "updated_at": "2024-06-15T12:00:00Z",
  "events": [
    {
```

```
{
  "event_id": "test-evt-initiated-001",
  "event_type": "payment_initiated",
  "amount": "7500.00",
  "currency": "INR",
  "timestamp": "2024-06-15T10:00:00Z",
  "received_at": "<server timestamp>"
},
{
  "event_id": "test-evt-processed-001",
  "event_type": "payment_processed",
  "amount": "7500.00",
  "currency": "INR",
  "timestamp": "2024-06-15T10:05:00Z",
  "received_at": "<server timestamp>"
},
{
  "event_id": "test-evt-settled-001",
  "event_type": "settled",
  "amount": "7500.00",
  "currency": "INR",
  "timestamp": "2024-06-15T12:00:00Z",
  "received_at": "<server timestamp>"
}
]
```

Understanding the 3 status Fields :

Field	Meaning
status	The overall / derived state of the transaction.Source of truth for UI display.
payment_status	Tracks specifically whether the payment was processed or failed.
settlement_status	Tracks whether funds were actually credited to the merchant.

TEST 8 : List Events (Pagination)

Purpose: Prove the event listing endpoint works with pagination controls.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/events?page=1&page_size=10
Pass Condition	total > 10,000 and items have exactly 10 entries.

Available Query Params

Parameter	Type	Default	Description
page	integer	1	Page number (1-indexed)
page_size	integer	—	Items per page
transaction_id	string	—	Filter to one transaction's events
merchant_id	string	—	Filter to one merchant's events
event_type	enum	—	payment_initiated / payment_processed / payment_failed / settled
sort_order	enum	desc	asc or desc (by timestamp)

Sample Response Body for TEST 8 (200 OK) :

```
{
  "total": 10169,
  "page": 1,
  "page_size": 10,
  "total_pages": 1017,
  "items": [
    {
      "event_id": "...",
      "event_type": "...",
      "transaction_id": "...",
      "merchant_id": "...",
      "amount": "...",
      "currency": "INR",
      "timestamp": "...",
      "received_at": "..."
    }
  ]
}
```

TEST 9 :Filter Events by Type

Purpose: Prove event type filtering is working correctly.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/events?event_type=payment_initiated&page_size=5
Pass Condition	Every item in items has event_type = " payment_initiated ". None should be payment_processed or settled.

TEST 10 :Filter Events by Transaction

Purpose: See all events that belong to your test transaction.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/events?transaction_id=test-txn-001
Pass Condition	Returns exactly 3 events — initiated, processed, settled — all for test-txn-001

TEST 11 :List Transactions with Filters

Purpose: Prove multi-filter transaction listing works.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?merchant_id=merchant_1&status=settled&page_size=5
Pass Condition	Every item has merchant_id = "merchant_1" and status = "settled".

Available Query Params

Parameter	Type	Default	Max	Description
merchant_id	string	—	—	Filter by merchant
status	enum	—	—	initiated / processed / failed / settled
date_from	ISO 8601 datetime	—	—	Transactions created on or after
date_to	ISO 8601 datetime	—	—	Transactions created on or before
page	integer	1	—	Page number
page_size	integer	20	100	Items per page
sort_by	enum	created_at	—	created_at / amount / status / updated_at
sort_order	enum	desc	—	asc or desc

TEST 12 :Date Range Filter

Purpose: Prove date range filtering on transactions.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?date_from=2024-06-01T00:00:00Z&date_to=2024-06-30T23:59:59Z&page_size=5
Pass Condition	All returned transactions have created_at within June 2024.

TEST 13 :Reconciliation Summary

Purpose: Prove the SQL aggregation engine produces the full financial picture..

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/summary
Pass Condition	total_transactions = 3800, by_merchant has 5 entries, by_status has 4 entries

Available Filters (all optional) :

Parameter	Description
merchant_id	Narrow to a single merchant
date_from	Start of date range (ISO 8601)
date_to	End of date range (ISO 8601)

Sample Response Body for TEST 13 (200 OK) :

```
{
  "generated_at": "<current timestamp>",
  "total_transactions": 3800,
  "total_amount": "95176625.69",
  "total_settled_amount": "66435431.65",
  "by_merchant": [
    {
      "merchant_id": "merchant_1",
      "merchant_name": "QuickMart",
      "total_transactions": 760,
      "total_amount": "19035325.14",
      "settled_count": 532,
      "settled_amount": "13285325.14",
      "processed_count": 76,
      "failed_count": 114,
      "initiated_count": 38,
      "settlement_rate_pct": 70.0
    }
  ],
  "by_status": [
    { "status": "settled", "count": 2660, "total_amount": "66435431.65" },
    { "status": "failed", "count": 570, "total_amount": "14276493.85" },
    { "status": "processed", "count": 380, "total_amount": "9517662.57" },
    { "status": "initiated", "count": 190, "total_amount": "4758858.28" }
  ],
  "by_date": [
    {
      "date": "2024-06-15",
      "total_transactions": 50,
```

```
    "total_amount": "1250000.00",
    "settled_count": 35,
    "settled_amount": "875000.00",
    "failed_count": 8,
    "processed_count": 7
  }
]
```

TEST 14 :Reconciliation Summary

Purpose: Prove per-merchant filtering works on the summary

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies
Pass Condition	by_merchant has exactly 1 entry with merchant_id = "merchant_1".

TEST 15 :Discrepancies (Anomaly Detection)

Purpose: Prove the system detects inconsistencies between payment state and settlement state

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies
Pass Condition	Response is 200 OK with a breakdown object containing all 3 keys and a discrepancies array.

Understanding Discrepancy Types:

Discrepancy Type	What It Means	Why It Matters
settled_after_failure	payment_status=failed but settlement_status=settled	Impossible state — potential fraud or accidental double credit. Highest severity.
processed_not_settled	payment_status=processed but settlement_status=pending	Merchant processed a payment but funds were never credited. Money is "stuck."
stale_initiated	Transaction has only 1 event (payment_initiated) and went nowhere	Payment was stuck — possibly a dropped message, timeout, or partner bug.

Sample Response Body for TEST 15 (200 OK) :

```
{
  "generated_at": "<current timestamp>",
  "total_discrepancies": 950,
  "breakdown": {
    "settled_after_failure": 0,
    "processed_not_settled": 380,
    "stale_initiated": 570
  },
  "discrepancies": [
    {
      "transaction_id": "txn-abc-123",
      "merchant_id": "merchant_2",
      "merchant_name": "TechStore",
      "amount": "15000.00",
      "currency": "INR",
      "payment_status": "processed",
      "settlement_status": "pending",
      "status": "processed",
      "discrepancy_type": "processed_not_settled",
      "discrepancy_description": "Payment processed but never settled. Merchant is owed funds.",
      "created_at": "...",
      "updated_at": "..."
    }
  ]
}
```

TEST 16 :Discrepancies (Anomaly Detection)

Purpose: Prove you can drill into a specific discrepancy category.

Step 16A : Processed but Not Settled

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies?discrepancy_type=processed_not_settled
Pass Condition	Every item has discrepancy_type = "processed_not_settled"

Step 16B :Stale Initiated

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies?discrepancy_type=stale_initiated
Pass Condition	Every item has discrepancy_type = "stale_initiated"

Step 16B :Settled After Failure (Edge Case)

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies?discrepancy_type=settled_after_failure
Pass Condition	Every item has discrepancy_type = "settled_after_failure"

TEST 17 :Filter Discrepancies by Merchant

Purpose: Prove you can investigate discrepancies for a specific merchant.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/reconciliation/discrepancies?
Pass Condition	Every item has merchant_id = "merchant_2"

TEST 18 :404 for Non-Existent Transaction

Purpose: Prove proper error handling — the API fails gracefully.

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions/this-transaction-does-not-exist
Pass Condition	Every item has merchant_id = "merchant_2"

Sample Response Body for TEST 18 (200 OK) :

```
{
  "detail": "Transaction 'this-transaction-does-not-exist' not found"
}
```

TEST 19 :Validation Error (422)

Purpose: Prove the API rejects malformed requests gracefully.

Field	Details
Method	POST
URL	https://setu-payment-service.vercel.app/events
Pass Condition	Status code is 422 and the detail array lists each individual validation error with the field location.
Body	(invalid — missing required fields and bad event_type)

Sample Request Body for TEST 19 :

```
{
  "event_id": "bad-event-001",
  "event_type": "unknown_type",
  "transaction_id": "txn-bad-001"
}
```

Sample Response Body for TEST 19 (200 OK) :

```
{
  "detail": [
    {
      "type": "enum",
      "loc": ["body", "event_type"],
      "msg": "Input should be 'payment_initiated', 'payment_processed', 'payment_failed' or 'settled'",
      "input": "unknown_type"
    },
    {
      "type": "missing",
      "loc": ["body", "merchant_id"],
      "msg": "Field required"
    }
  ]
}
```

TEST 20 :Sort & Pagination Edge Cases

Purpose: Prove sorting and page boundaries work.

Step 20A :Sort by Amount Ascending

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?sort_by=amount&sort_order=asc&page_size=5
Pass Condition	First item has the lowest amount in the dataset

Step 20B :Large Page Number (Beyond Last Page)

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?page=9999&page_size=20
Pass Condition	Returns a 200 OK with "items": []. It successfully returns an empty array instead of crashing with an error.

Step 20C :Exceed Max Page Size

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?page_size=100
Pass Condition	Returns exactly 100 items in the "items" array (the maximum allowed size per page).

Step 20D :Exceed Max Page Size

Field	Details
Method	GET
URL	https://setu-payment-service.vercel.app/transactions?page_size=101
Pass Condition	422 with a validation error about the maximum

Summary of All Test Cases

#	Endpoint	Method	What It Proves
1	/health	GET	Service is alive
2	/events	POST	New event → ingestion_status: created
3	/events	POST	Idempotency: same event → already_processed
4	/events	POST	payment_processed event accepted
5	/events	POST	settled event accepted
6a	/events	POST	payment_initiated for new txn
6b	/events	POST	payment_failed → failure path works
7	/transactions/test-txn-001	GET	State machine correct: status=settled, 3 events in history
8	/events?page=1&page_size=10	GET	Pagination: total > 10,000, 10 items returned
9	/events?event_type=payment_initiated	GET	Event type filter works
10	/events?transaction_id=test-txn-001	GET	Transaction filter: exactly 3 events
11	/transactions?merchant_id=merchant_1&status=settled	GET	Multi-filter: all items match both filters
12	/transactions?date_from=...&date_to=...	GET	Date range filter works
13	/reconciliation/summary	GET	SQL aggregation: 3800 txns, 5 merchants, 4 statuses
14	/reconciliation/summary?merchant_id=merchant_1	GET	Summary filtered to 1 merchant
15	/reconciliation/discrepancies	GET	Anomaly detection: breakdown has all 3 types
16a	/reconciliation/discrepancies?discrepancy_type=processed_not_settled	GET	Filtered discrepancies
16b	/reconciliation/discrepancies?discrepancy_type=stale_initiated	GET	Stale payments detected
16c	/reconciliation/discrepancies?discrepancy_type=settled_after_failure	GET	Zero fraudulent states (clean data)
17	/reconciliation/discrepancies?merchant_id=merchant_2	GET	Merchant-specific anomaly view
18	/transactions/this-transaction-does-not-exist	GET	404 error — graceful failure
19	/events (bad body)	POST	422 error — validation catches bad

			data
20a	/transactions?sort_by=amount&sort_order=asc	GET	Sorting works
20b	/transactions?page=9999	GET	Empty page — no crash beyond last page
20c	/transactions?page_size=100	GET	Max page size respected
20d	/transactions?page_size=101	GET	Exceeding max page size returns 422

Key Concepts for Non-Technical Reviewers

What is event_id vs transaction_id?

Field	Description	Simple Analogy
event_id	A unique ID assigned to one specific event message.	Like a receipt number.
transaction_id	Links multiple related events together as part of the same payment journey.	Like an order number.

What is Idempotency?

If your network drops mid-request and you retry, the server may receive the same event twice. Idempotency means the second send is harmless — the system recognizes it's a duplicate and ignores it safely. This is non-negotiable in fintech. Money must not be debited twice.

What does ingestion_status tell me?

ingestion_status	Meaning
"created"	Brand-new event that was stored successfully.
"already_processed"	This event_id has already been received and processed. The duplicate can be safely ignored.

What is Reconciliation?

Every night (or on-demand), finance teams compare what the payment processor says happened vs. what the settlement system recorded. Any mismatch = a discrepancy that needs human review.

Useful URLs to Open in Browser

URL	What to Expect
https://setu-payment-service.vercel.app/	Visual dashboard — charts, live stats, transaction table
https://setu-payment-service.vercel.app/redoc	Developer Guide — event glossary, state machine, error codes, endpoint cards
https://setu-payment-service.vercel.app/docs	Swagger UI — interactive API playground
https://setu-payment-service.vercel.app/health	Raw JSON health check